

Idea

Project Name:
Idea Diligence Agent

Elevator Pitch:

An AI agent that researches product ideas, competitors, markets, pricing, and revenue, challenges assumptions, and delivers evidence-backed GO, MODIFY, or KILL decisions.

Elevator Pitch: Edmund's suggestion

An autonomous AI agent that takes a raw product idea, researches competitors, markets, pricing, and revenue in the background, challenges your assumptions, and delivers an evidence-backed GO, MODIFY, or KILL decision, so you spend hours building, not Googling.

Edmund's points

Below is Edmund's refined project specification, architecture blueprint, and development guide. This synthesizes Gokul's feedback into an actionable, development-ready plan designed for all team members—including junior developers—to read, understand, and start building immediately.

1. Executive Summary & Response to Gokul's Architectural Feedback

I completely agree with Gokul's key architectural insights. His review pushed our thinking from a rigid, linear script into a robust, autonomous agent system. Specifically:

- **Canonical Structured State:** Instead of treating state as an ad-hoc database or simple chat history, the investigation centers on a single canonical data structure—the DiligenceState. Every specialist agent reads from it and contributes structured updates.
- **Adaptive Research Loop:** Instead of running Problem → Competition → Economics in a single fixed line, the Orchestrator runs an adaptive loop. At each step, it identifies the single highest-value unknown and dispatches the appropriate specialist to resolve it.
- **Session Isolation:** Each idea is investigated in an isolated session (ResearchSession) with a fresh DiligenceState, generating a persistent final report and cleanly disposing active memory so that batch processing (e.g. 10 ideas) is trivial and state leaks never happen.
- **Evidence Classification:** We do not treat all web data equally. Findings are strictly classified into FACT, ASSUMPTION, INFERENCE, and UNKNOWN, giving our orchestrator and users clear confidence boundaries.
- **Lean Development Philosophy:** We will not build a 40-field schema before testing real queries. We start with minimal core fields, validate them end-to-end with the Strands SDK and Amazon Bedrock, and let real research workflows guide model expansion.

2. Problem Statement & User Experience

Why We Are Building This:

- Founders, indie hackers, and product managers waste dozens of hours manually Googling competitors, reading pricing tiers, guessing addressable market sizes, and compiling spreadsheets.
- Founders are prone to severe confirmation bias—searching only for evidence that validates their enthusiasm while missing fatal red flags or strong incumbent moats.
- Most people either build for months without basic market diligence, or prematurely kill viable concepts due to lack of competitive clarity.

The User Experience (Zero Babysitting):

- **1. Submission:** The user enters a plain-English product concept (e.g., 'An AI tool that tracks inventory waste for independent restaurants').
- **2. Autonomous Research:** The agent triggers an autonomous investigation in the background. The user does not need to answer 20 tedious prompt questions.
- **3. Multi-Turn Diligence:** The agent searches the web, parses competitor pricing tables, identifies target buyer segments, and evaluates unit economics.

- **4. Assumption Challenge:** The agent stress-tests the founder's assumptions against hard market evidence and competing alternatives.
- **5. Decision Dossier:** The agent delivers a clear, evidence-backed verdict (GO, MODIFY, or KILL) alongside a structured diligence dossier.

3. System Architecture & The Diligence Loop

The system architecture consists of four primary components operating over an isolated session:

Component A: The Canonical Research State (DiligenceState)

DiligenceState is our single source of truth for an investigation. It contains:

- **idea:** Raw text description and initial user inputs.
- **problem:** Analysis of customer pain points, severity, and existing manual workarounds.
- **customers:** List of target personas, buyer profiles, and willingness-to-pay indicators.
- **competitors:** Direct competitors, indirect substitutes, pricing tiers, and feature gaps.
- **economics:** Pricing model options, unit margins, and estimated market size (TAM).
- **evidence:** Audit log of all gathered evidence categorized by epistemic type.
- **unknowns:** List of remaining high-priority uncertainties that require research.
- **research_history:** Step-by-step record of actions, agent invocations, and queries.
- **verdict:** Final decision (GO, MODIFY, KILL) with confidence score and reasoning.

Evidence Classification Hierarchy

- **FACT:** Verified, sourced empirical facts (e.g., 'Competitor X charges \$49/mo on their public pricing page'). Must include a URL or citation.
- **ASSUMPTION:** Unverified hypotheses or claims made by the user (e.g., 'Small restaurants will readily adopt an iPad app for waste tracking').
- **INFERENCE:** Logical deductions formed by connecting verified facts (e.g., 'Because 3 competitors offer tiered pricing above \$50, the market has willingness to pay in this range').
- **UNKNOWN:** Critical missing data that prevents confident decision-making (e.g., 'Average customer churn rate in this vertical').

Component B: The Adaptive Orchestrator

The Orchestrator acts as the lead investigator. Instead of blindly executing agents in sequence, it follows an adaptive loop:

- 1. Inspect current DiligenceState and extract the list of critical UNKNOWNs.

- 2. Prioritize the single most valuable unknown to resolve next.
- 3. Dispatch the specialist agent best suited to answer it (Problem, Competition, or Economics).
- 4. Ingest the specialist's structured findings and merge them into DiligenceState.
- 5. Evaluate stopping conditions: If evidence is sufficient to make a confident recommendation OR maximum cycle limit (e.g., 5 cycles) is reached, trigger the Verdict Engine; otherwise, repeat.

Component C: The Three Specialist Research Agents

- **1. Problem Analysis Agent:** Investigates who experiences the problem, how acute the pain is, what manual workarounds exist today, and whether active demand signals exist.
- **2. Competition Analysis Agent:** Scrapes search results for direct competitors and indirect alternatives, finds published pricing plans, uncovers customer complaints on review sites, and pinpoints market whitespace.
- **3. Economics Analysis Agent:** Evaluates pricing models (subscription, usage, tiered), customer acquisition cost (CAC) vs. lifetime value (LTV) risks, and total addressable market feasibility.

Component D: Research Tools

- **search_web:** Live web search tool (DuckDuckGo search) enabling agents to find live websites, press releases, and reviews without requiring paid API keys.
- **read_webpage:** HTTP fetch and markdown extractor tool that retrieves webpage content and strips noise for LLM consumption.
- **save_finding:** A structured recording tool that forces agents to tag findings as FACT, ASSUMPTION, INFERENCE, or UNKNOWN with citation URLs.

4. The Decision Verdict Framework

At the end of an investigation, the agent synthesizes all evidence and renders one of three definitive verdicts:

- **GO:** Strong problem-solution fit, verified customer pain, identifiable competitive gap, defensible unit economics, and validated key assumptions. The recommendation is to proceed to MVP development.
- **MODIFY:** The overarching problem is real, but fatal obstacles exist in the current formulation (e.g., crowded direct competition, poor pricing power, or wrong customer segment). The agent outputs exact pivot recommendations (e.g., 'Pivot from independent cafes to multi-location franchises').
- **KILL:** The idea faces insurmountable headwinds: entrenched free incumbents, negligible willingness to pay, severe regulatory hurdles, or an unverified core problem. The agent provides clear evidence saving the founder months of wasted effort.

5. Technology Stack & AWS Alignment

To maximize our technical score in the AWS Strands Agents SDK Hackathon, our implementation specifically leverages:

- **Agent Framework:** Strands Agents SDK (Python) utilizing both the Agents-as-Tools pattern for rapid specialist delegation and the Graph pattern for cyclic feedback loops with bounded execution.
- **Foundation Model:** Amazon Bedrock running Anthropic Claude Sonnet, giving us industry-leading reasoning, accurate structured JSON outputs, and fast execution within AWS.
- **Data Modeling:** Pydantic v2 data models for rigorous schema enforcement, type safety, and clean JSON serialization across all agent interfaces.
- **Interface:** Clean Python CLI with rich terminal formatting for local execution, with modular endpoints designed for rapid web UI or AWS AgentCore deployment.

6. Development Roadmap for the Team & Junior Developers

Here is our step-by-step build sequence for the hackathon sprint:

- **Phase 1 - Walking Skeleton:** Set up project skeleton, dependencies, Pydantic data models (models.py), custom research tools (tools.py), and test Bedrock connectivity. [COMPLETED]
- **Phase 2 - Multi-Agent Loop:** Implement the specialist agents (Problem, Competition, Economics) with targeted system prompts and wire the Orchestrator loop using the agents-as-tools coordinator.
- **Phase 3 - Verdict & Synthesis Engine:** Refine the evidence classifier (FACT/ASSUMPTION/INFERENCE/UNKNOWN), add confidence scoring, and build the GO/MODIFY/KILL verdict generator.
- **Phase 4 - Demo, Polish & Submission:** Create interactive CLI/UI demo, record the 5-minute hackathon walkthrough video, write README documentation, and submit to Devpost.

7. Golden Rules for Developers Working on This Codebase

- **1. Workflow Shapes the Model:** Do not add fields to DiligenceState until a real research run proves that an agent needs to produce or consume that data.
- **2. Mandatory Provenance:** Never allow an agent to label a statement as a FACT without an accompanying source URL or reference.

- **3. Separation of Concerns:** Each specialist agent must focus strictly on its designated domain. Problem does not do pricing math; Economics does not do competitor discovery.
 - **4. Bounded Execution:** Every loop must have a hard execution cap (e.g. max 5 cycles) so the agent never hangs or consumes unbounded tokens.
-

Gokul's Points

Yes. **Your hypothesis is architecturally sound**, and I think you're identifying an important requirement before jumping into "three sub-agents."

What you're describing is essentially a **shared research workspace / research state** that the orchestrator continuously updates as the investigation progresses.

And interestingly, this maps quite closely to capabilities Strands now provides: its multi-agent patterns support shared state, and Graph supports conditional/cyclic workflows and feedback loops. ([Strands Agents SDK](#))

I would refine your idea slightly

Don't think of it initially as:

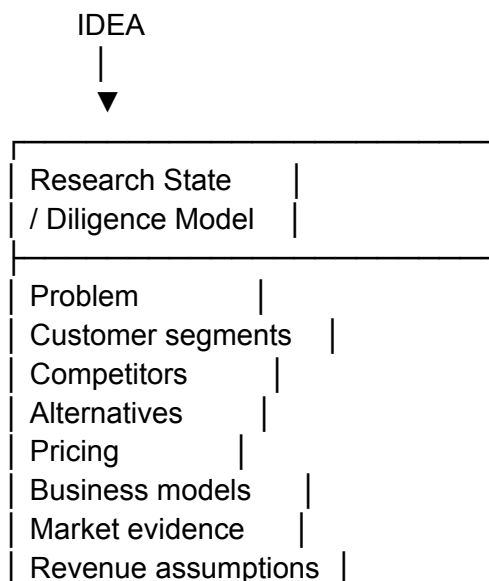
"The orchestrator needs a database."

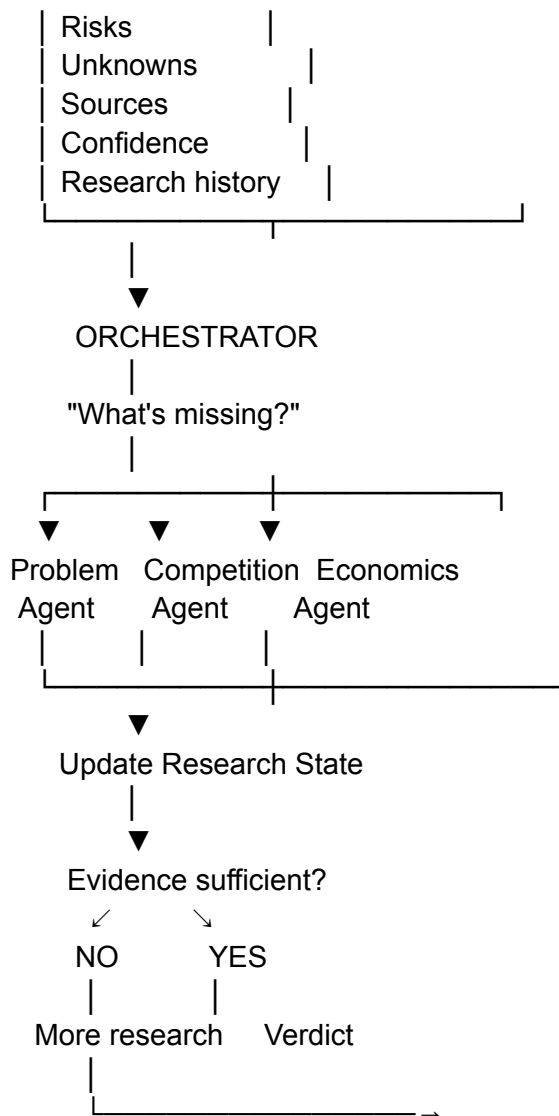
Think:

"The investigation needs a canonical structured state that represents everything currently known about the idea."

That distinction matters.

Something like:





That's much more powerful than simply passing the output of Agent A into Agent B.

The key concept: a canonical "Diligence Model"

I'd actually give this thing a name inside your architecture:

DiligenceState

or

ResearchState

For example, conceptually:

```
class DiligenceState:
    idea: Idea
    problem: ProblemAnalysis
    customers: list[CustomerSegment]
    competitors: list[Competitor]
    alternatives: list[Alternative]
    business_models: list[BusinessModel]
    pricing: list[PricingEvidence]
    economics: EconomicModel
    assumptions: list[Assumption]
    evidence: list[Evidence]
    unknowns: list[Unknown]
    research_history: list[ResearchAction]
    verdict: Verdict | None
```

I'm **not suggesting you build exactly this class yet**. I'm showing you the conceptual model.

The important thing is that every specialist isn't producing an isolated essay.

They're producing **updates to a common representation of the investigation**.

Then your orchestrator becomes much more interesting

The orchestrator doesn't need to ask:

"Run all three agents."

Instead it asks:

"Given what we currently know, what is the most valuable next investigation?"

For example:

Initial state

Problem:
partially understood

Competition:
unknown

Economics:
unknown

Unknowns:
- Who is the actual customer?
- Are there existing competitors?
- What do they charge?

Orchestrator:

We don't understand the customer sufficiently.

→ call **Problem Agent**

State becomes:

Problem:
✓ SMB restaurants
✓ inventory wastage
✓ manual spreadsheet workflow

Unknowns:
- Are existing SaaS products solving this?
- What do they charge?

Orchestrator:

Now competition is the highest-value unknown.

→ call **Competition Agent**

Then:

Competition:
✓ 4 direct competitors
✓ 7 adjacent alternatives
✓ pricing found for 3

Unknowns:
- Is the proposed pricing viable?

→ Economics Agent.

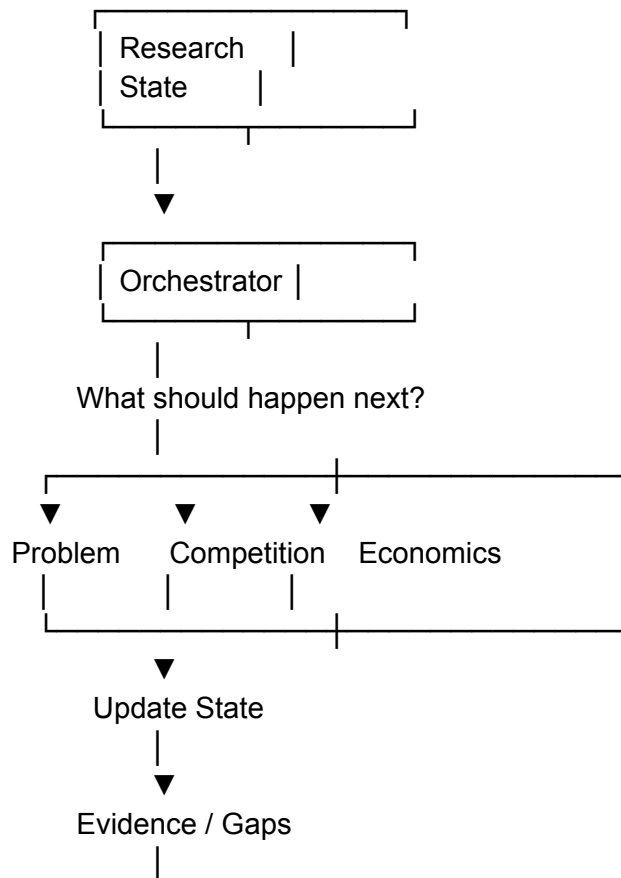
That is **genuine adaptive research**.

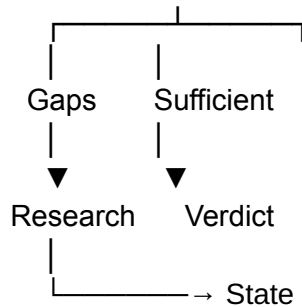
And this is where your original diagram gets upgraded

Your original:

Problem
Competition
Economics
↓
Evidence

becomes:





That is the architecture I would take to Edmund.

One thing I would change in your thinking

You said:

"update it or delete it to be open for new idea researching"

Yes — but I'd make the lifecycle:

Idea A



ResearchState A



Complete



Persist report



Dispose active state



Idea B

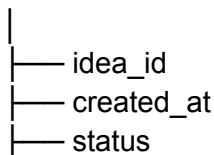


Fresh ResearchState B

Don't have one giant mutable state object containing every idea.

Instead:

ResearchSession



```
|— research_state
|— evidence
|— history
|— final_report
```

Then:

Idea #1 → Session #1

Idea #2 → Session #2

Idea #3 → Session #3

That gives you isolation and makes your eventual "upload 10 ideas" workflow much cleaner.

Strands itself has session-management concepts for stateful single- and multi-agent systems, including orchestrator state, agent results, shared state, and execution history. ([Strands Agents SDK](#))

And here's the really interesting part

Your "data model" should probably distinguish **facts**, **assumptions**, **hypotheses**, and **unknowns**.

This is critical for Idea Diligence.

For example:

FACT

Competitor X charges \$49/month.

Source: competitor website.

FACT

Competitor X has approximately 10,000 customers.

Source: public interview.

ASSUMPTION

Our product could charge \$39/month.

INFERENCE

The market appears willing to pay for this category.

UNKNOWN

Customer acquisition cost.

UNKNOWN

Whether small businesses would switch from spreadsheets.

Then the orchestrator can reason over the **unknowns**.

That gives you:

Research isn't "search until the report looks complete."

It's:

Search until the important uncertainties are reduced enough to make a decision.

That is a much stronger architecture.

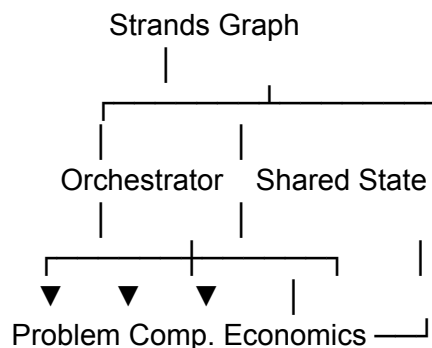
And this aligns extremely well with Strands

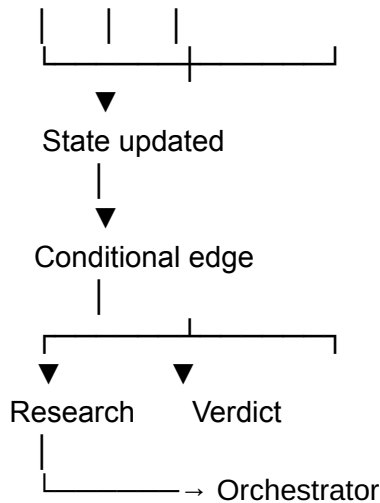
I checked the current Strands documentation because you're specifically designing around its multi-agent architecture.

Strands' Graph pattern supports **cyclic topologies, conditional routing, iterative refinement, shared state, and bounded execution**, which maps very naturally to your proposed research loop. ([Strands Agents SDK](#))

It also supports shared state across Graph/Swarm agents, while keeping state/configuration separate from what is placed into the model's prompt. ([Strands Agents SDK](#))

So conceptually:





And Strands Graph explicitly supports feedback loops such as this, with execution limits to prevent an unbounded cycle. ([Strands Agents SDK](#))

But don't over-engineer the state yet

You **don't yet know the final schema**.

Start with maybe:

Idea
Problem
Customers
Competitors
Economics
Evidence
Assumptions
Unknowns
Verdict

Then build one real research run.

You'll discover:

"Oh, Competition needs pricing evidence."

Then add it.

You'll discover:

"Revenue assumptions need provenance."

Add it.

You'll discover:

"This finding contradicts another finding."

Add contradiction handling.

Gokul's Points

Yes. **Your hypothesis is architecturally sound**, and I think you're identifying an important requirement before jumping into "three sub-agents."

What you're describing is essentially a **shared research workspace / research state** that the orchestrator continuously updates as the investigation progresses.

And interestingly, this maps quite closely to capabilities Strands now provides: its multi-agent patterns support shared state, and Graph supports conditional/cyclic workflows and feedback loops. ([Strands Agents SDK](#))

I would refine your idea slightly

Don't think of it initially as:

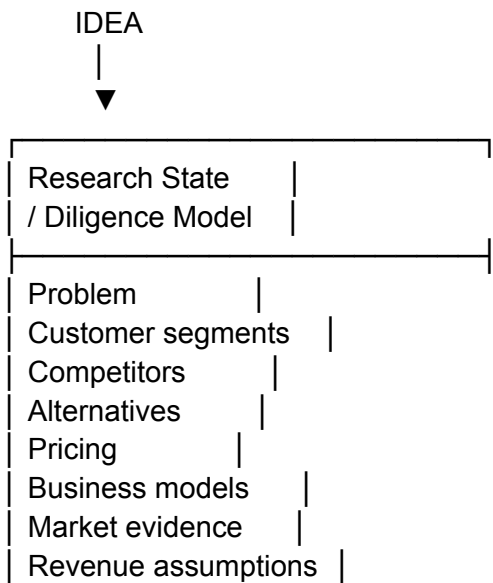
"The orchestrator needs a database."

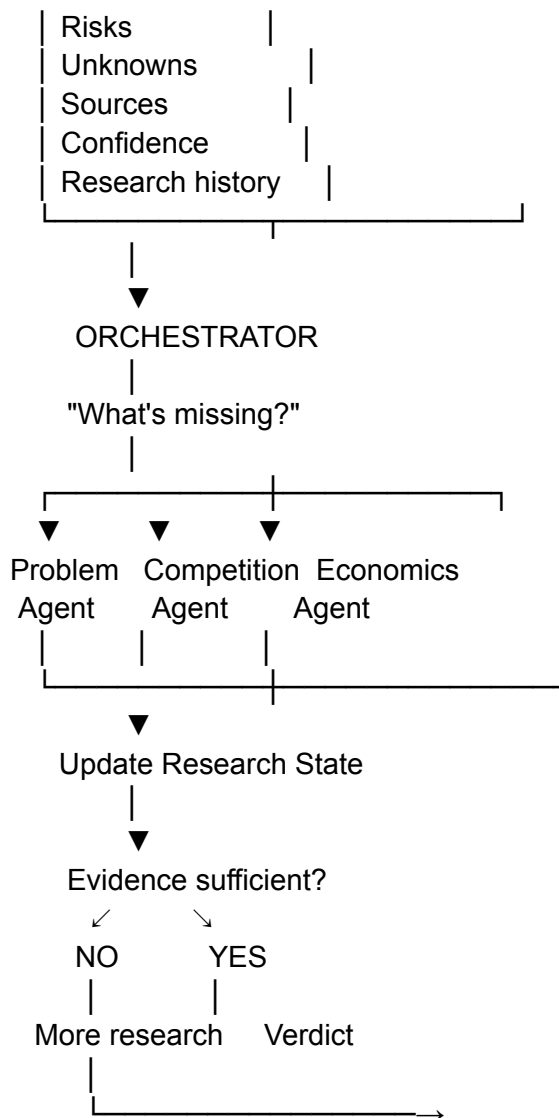
Think:

"The investigation needs a canonical structured state that represents everything currently known about the idea."

That distinction matters.

Something like:





That's much more powerful than simply passing the output of Agent A into Agent B.

The key concept: a canonical "Diligence Model"

I'd actually give this thing a name inside your architecture:

DiligenceState

or

ResearchState

For example, conceptually:

```
class DiligenceState:
    idea: Idea
    problem: ProblemAnalysis
    customers: list[CustomerSegment]
    competitors: list[Competitor]
    alternatives: list[Alternative]
    business_models: list[BusinessModel]
    pricing: list[PricingEvidence]
    economics: EconomicModel
    assumptions: list[Assumption]
    evidence: list[Evidence]
    unknowns: list[Unknown]
    research_history: list[ResearchAction]
    verdict: Verdict | None
```

I'm **not suggesting you build exactly this class yet**. I'm showing you the conceptual model.

The important thing is that every specialist isn't producing an isolated essay.

They're producing **updates to a common representation of the investigation**.

Then your orchestrator becomes much more interesting

The orchestrator doesn't need to ask:

"Run all three agents."

Instead it asks:

"Given what we currently know, what is the most valuable next investigation?"

For example:

Initial state

Problem:
partially understood

Competition:
unknown

Economics:
unknown

Unknowns:
- Who is the actual customer?
- Are there existing competitors?
- What do they charge?

Orchestrator:

We don't understand the customer sufficiently.

→ call **Problem Agent**

State becomes:

Problem:
✓ SMB restaurants
✓ inventory wastage
✓ manual spreadsheet workflow

Unknowns:
- Are existing SaaS products solving this?
- What do they charge?

Orchestrator:

Now competition is the highest-value unknown.

→ call **Competition Agent**

Then:

Competition:
✓ 4 direct competitors
✓ 7 adjacent alternatives
✓ pricing found for 3

Unknowns:
- Is the proposed pricing viable?

→ Economics Agent.

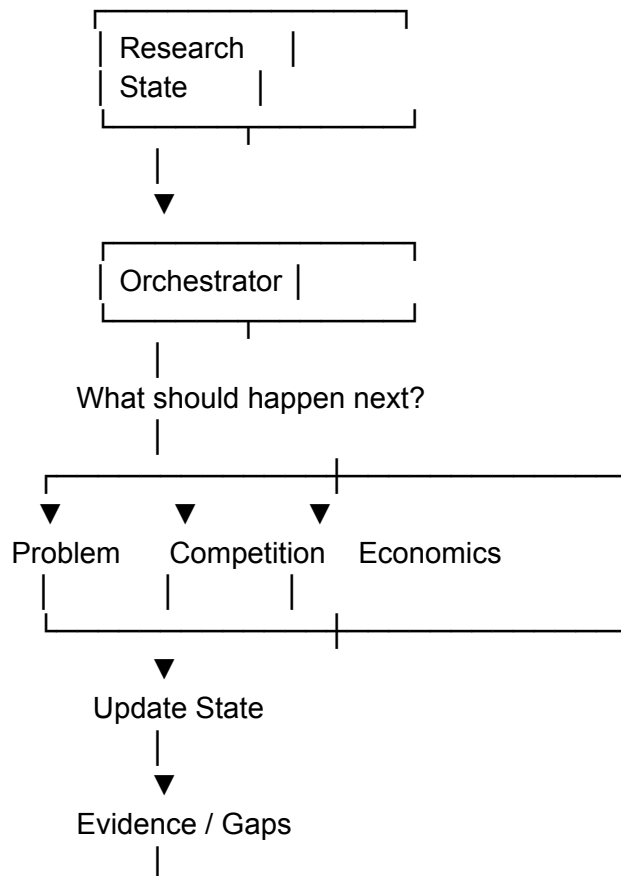
That is **genuine adaptive research**.

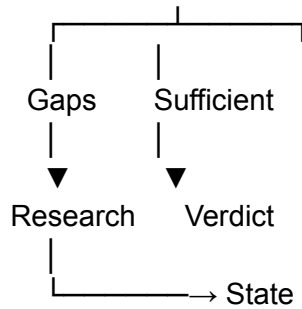
And this is where your original diagram gets upgraded

Your original:

Problem
Competition
Economics
↓
Evidence

becomes:





That is the architecture I would take to Edmund.

One thing I would change in your thinking

You said:

"update it or delete it to be open for new idea researching"

Yes — but I'd make the lifecycle:

```
graph TD; A[Idea A] --> B[ResearchState A]; B --> C[Complete]; C --> D[Persist report]; D --> E[Dispose active state]; E --> F[Idea B]; F --> G[Fresh ResearchState B];
```

Don't have one giant mutable state object containing every idea.

Instead:

```
ResearchSession
|
|— idea_id
|— created_at
|— status
```

```
|— research_state
|— evidence
|— history
|— final_report
```

Then:

Idea #1 → Session #1

Idea #2 → Session #2

Idea #3 → Session #3

That gives you isolation and makes your eventual "upload 10 ideas" workflow much cleaner.

Strands itself has session-management concepts for stateful single- and multi-agent systems, including orchestrator state, agent results, shared state, and execution history. ([Strands Agents SDK](#))

And here's the really interesting part

Your "data model" should probably distinguish **facts**, **assumptions**, **hypotheses**, and **unknowns**.

This is critical for Idea Diligence.

For example:

FACT

Competitor X charges \$49/month.

Source: competitor website.

FACT

Competitor X has approximately 10,000 customers.

Source: public interview.

ASSUMPTION

Our product could charge \$39/month.

INFERENCE

The market appears willing to pay for this category.

UNKNOWN

Customer acquisition cost.

UNKNOWN

Whether small businesses would switch from spreadsheets.

Then the orchestrator can reason over the **unknowns**.

That gives you:

Research isn't "search until the report looks complete."

It's:

Search until the important uncertainties are reduced enough to make a decision.

That is a much stronger architecture.

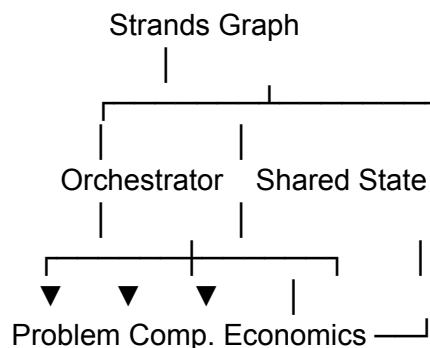
And this aligns extremely well with Strands

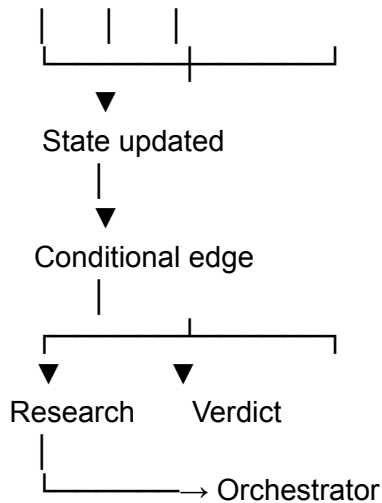
I checked the current Strands documentation because you're specifically designing around its multi-agent architecture.

Strands' Graph pattern supports **cyclic topologies, conditional routing, iterative refinement, shared state, and bounded execution**, which maps very naturally to your proposed research loop. ([Strands Agents SDK](#))

It also supports shared state across Graph/Swarm agents, while keeping state/configuration separate from what is placed into the model's prompt. ([Strands Agents SDK](#))

So conceptually:





And Strands Graph explicitly supports feedback loops such as this, with execution limits to prevent an unbounded cycle. ([Strands Agents SDK](#))

But don't over-engineer the state yet

You **don't yet know the final schema**.

Start with maybe:

Idea
Problem
Customers
Competitors
Economics
Evidence
Assumptions
Unknowns
Verdict

Then build one real research run.

You'll discover:

"Oh, Competition needs pricing evidence."

Then add it.

You'll discover:

"Revenue assumptions need provenance."

Add it.

You'll discover:

"This finding contradicts another finding."

Add contradiction handling.

Let the research workflow shape the data model.

Don't spend half a day designing a 40-field Pydantic model before the first agent has actually researched an idea.
